Absolutely. For your situation—**one project worked on locally on two devices with GitHub as the remote**—you mainly need a relatively small set of commands.

The basic routine

This is the workflow you'll use most often:

1. git pull
   ↓
2. work on your code
   ↓
3. git status
   ↓
4. git add .
   ↓
5. git commit -m "Description of changes"
   ↓
6. git push

1. `git pull`

Get changes from GitHub onto your computer:

git pull

Use it: Usually when you start working, especially if you've worked on the other computer since your last session.

---

2. `git status`

See what's going on:

git status

This tells you:

- which files have changed
- which files are untracked
- whether you have files staged for a commit
- whether your local branch is ahead/behind GitHub

Use it frequently. It's one of the most useful Git commands.

---

3. `git add`

Tell Git which changes should go into your next commit:

git add filename.py

or, to add everything in the current directory:

git add .

Important: **`git add` does not commit anything.**

It moves changes into the **staging area**:

Changed files
↓
git add
↓
Staging area

---

4. `git commit`

Create a permanent snapshot of the staged changes:

git commit -m "Improve SAXS fitting"

The message should briefly describe what you changed.

For example:

git commit -m "Add background subtraction"
git commit -m "Fix weighted averaging"
git commit -m "Add vesicle fitting model"

A commit is **local**. It has not yet gone to GitHub.

git add
↓
git commit
↓
LOCAL Git history

---

5. `git push`

Send your local commits to GitHub:

git push

The first time you push a branch, you may need:

git push -u origin main

After that, normally just:

git push

The commands you will use most

Command	Meaning
`git status`	What's changed?
`git pull`	Get changes from GitHub
`git add .`	Stage all changes
`git add file.py`	Stage one file
`git commit -m "..."`	Create a local commit
`git push`	Upload commits to GitHub
`git log`	Show commit history

Working between two computers

Suppose you work on **Computer A**:

git pull

```
# make changes
git add .
git commit -m "Add fitting routine"
git push
```

Then move to **Computer B**:

```
git pull
# make changes
git add .
git commit -m "Fix fitting parameters"
git push
```

Then back to A:

```
git pull
```

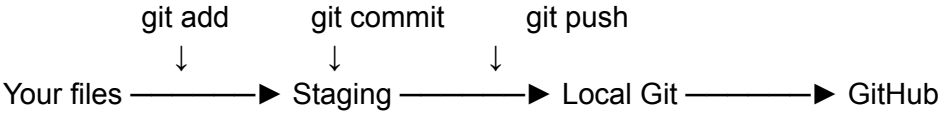
And so on.

The important habit is:

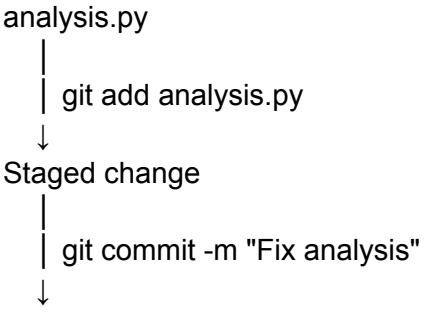
> **Pull before you start working; push when you've finished a meaningful piece of work.**

Understanding the three levels

This is probably the most important conceptual thing to understand:



For example, you modify `analysis.py`:



Local commit
  │
  │ git push
  ↓
GitHub

And in the other direction:

GitHub
  │
  │ git pull
  ↓
Your local Git repository
  │
  ↓
Your working files

Useful commands for checking things

See your remote

git remote -v

You'll see something like:

origin https://github.com/riko-hilger/BM29_data_and_analysis_2026.git

`origin` is simply the name Git gives your GitHub repository.

See your branches

git branch

Your current branch has a `*`:

* main

See commit history

git log

A shorter version:

git log --oneline

For example:

a31f82c Add background subtraction
7bd91a2 Fix SAXS averaging
41d8c73 Initial commit

Undoing things

These are useful, but be a little more careful with them.

You changed a file but don't want the changes

git restore filename.py

This throws away your **uncommitted** changes to that file.

For example:

git restore analysis.py

⚠️ Don't use this if you might want the changes back.

You staged a file but want to unstage it

git restore --staged filename.py

The file's changes remain on your computer; they are simply removed from the staging area.

`.gitignore`

You should probably have a `.gitignore` in your analysis repository.

For example:

__pycache__/
*.pyc
.venv/
.vscode/

Then:

git add .

won't add those files.

For a scientific analysis repository, you may also want to exclude large experimental data files, depending on how you want to manage your SAXS data.

One routine I recommend for you

When starting work on either computer:

git pull

Then work normally.

When you've reached a sensible stopping point:

git status
git add .
git commit -m "Describe what I changed"
git push

You don't need to commit after every tiny change. **A commit should represent a meaningful state of the project**—for example, "background subtraction working" or "add asymmetric vesicle model."

And one particularly useful habit:

git status

If you're ever unsure what Git is doing, **run `git status` first**. It is often the best diagnostic command.